UNIVERSITY OF
CAMBRIDGE
Department of Computer
Science and Technology

May 2026

Query-based Dependent Type Elaborator

Submitted in partial fulfilment of the requirements for the
Computer Science Tripos, Part II

Declaration

I, <Author> of <College>, being a candidate for the Computer Science Tripos, Part II, hereby declare that this dissertation and the work described in it are my own work, unaided except as may be specified below, and that the dissertation does not contain material that has already been used to any substantial extent for a comparable purpose.

Signed:
Date: May 2026

Proforma

Candidate Number	TODO
Title	Query-based Dependent Type Elaborator
Examination	Computer Science Tripos — Part II, 2026
Word Count	8,846[1]
Code Line Count	10,582[2]
Project Originator	The candidate
Project Supervisor	Dr Jon Sterling

0.1 Original aims of the project

Dependently typed languages — such as Lean, Agda, and Rocq — require type checkers that execute arbitrary programs at compile time. Most existing implementations operate in batch mode, re-checking entire files after each edit. This project set out to investigate whether a query-based, incremental approach could reduce the latency of elaboration in an interactive setting, by applying the "Build Systems à la Carte" framework to dependent type elaboration.

0.2 Work completed

TODO: fill in once evaluation is complete.

0.3 Special difficulties

None.

[1] Computed via `typst compile count.typ` and `pdftotext` with the bibliography stripped.
[2] Computed with `cloc`, excluding `.lake`, references, and stdlib examples.

Abstract

Dependently typed languages require expensive type checking, yet most elaborators operate in batch mode, re-checking entire files after each edit. This project implements an incremental, query-based elaborator for a dependently typed language supporting dependent function types, universe polymorphism, and inductive types with recursors. The elaborator is built on a formalisation of the "Build Systems à la Carte" framework in Lean 4, using a Shake-based incremental build system to memoise elaboration queries across edits. We evaluate the system on a standard library and conversion checking benchmarks, comparing incremental re-elaboration against batch re-elaboration.

Acknowledgements

TODO

Contents

Chapter 1

Introduction

1.1 Motivation

Dependently typed languages such as Lean [1], Agda [2], and Rocq [3] serve as both programming languages and proof assistants. Unlike ordinary compilers, their type checkers must execute arbitrary programs at compile time — normalising terms, checking definitional equality, and resolving implicit arguments. This process, called *elaboration*, is the most expensive phase of compilation. In an interactive setting — where a proof assistant is expected to provide rapid feedback on each keystroke — re-elaboration latency is a bottleneck.

Existing systems offer limited forms of incrementality. Lean, Agda, and Rocq all support *file-level* caching: if a file has not changed, its previously compiled result is reused. Within a single file, Lean (since version 4.8) saves snapshots of elaboration state and resumes from the most recent valid checkpoint; Agda's `--caching` flag reuses results for the unchanged prefix of declarations; and coq-lsp re-checks from the first modified sentence onward. All three treat the file as a sequence and reprocess a suffix. If definition 5 of 200 changes, definitions 6 through 200 are re-checked — even if none of them depend on definition 5.

The central observation is that most of this re-checking is redundant. If one definition changes but its type is unaffected, then definitions that depend only on its type need not be re-checked at all. What is needed is a way to track which results depend on which, and recompute only what has actually been invalidated. Unlike in a conventional compiler, where downstream code depends only on type signatures, a dependently typed elaborator may need to reduce through definition *bodies* during conversion checking — so the dependency structure is finer-grained and discovered dynamically during elaboration.

Query-based incrementality has been applied to conventional compilers: rust-analyzer [4] uses the Salsa framework [5] to provide incremental type checking for Rust, and the Roslyn compiler does similar work for C#. In these languages, the compilation boundary is the type signature — changing a function body without changing its type cannot affect downstream checking. Fredriksson's Sixten (later Sixty) [6] is the only prior work to apply query-based incrementality to a dependently typed language. It demonstrates that the approach is viable, but does not formalise the underlying build system or verify that incremental results agree with batch elaboration. No mainstream proof assistant — Lean, Agda, or Rocq — uses query-based incrementality.

This is a well-studied problem in a different guise: it is exactly what build systems do. The framework of Mokhov et al. [7] shows that build systems such as Make, Shake, and Bazel are instances of a single polymorphic type, parameterised by the class of effects a task is allowed to perform. *Shake* in particular supports dynamic dependencies — dependencies discovered during execution rather than declared up front — and *early cutoff* — skipping dependents when a recomputed result is unchanged. These are precisely the properties needed: the elaborator discovers which definitions it must unfold only as it runs, and early cutoff prevents unnecessary re-elaboration when an intermediate result has not changed.

This project applies that framework to elaboration. The elaborator decomposes its work into fine-grained queries — "what is the type of this constant?", "what are the errors in this definition?" — each of which is memoised by a Shake-based build system. When a definition changes, only the queries that transitively depend on it are reconsidered; and if the result of a recomputed query is unchanged, its dependents are not recomputed at all. The implementation is written in Lean 4, built on a formalisation of the build system framework in the same language.

1.2 Contributions

- An incremental, query-based elaborator for a dependently typed language, supporting dependent function types, universe polymorphism, and inductive types with recursors — built on a formalisation of the "Build Systems à la Carte" framework in Lean 4.
- An evaluation comparing incremental re-elaboration against batch re-elaboration on a standard library.

1.3 Overview

Section 2 introduces the background material: the build system framework of Mokhov et al. and our refinements to it, the dependent type theory we implement, bidirectional type checking, normalisation by evaluation, and related work. Section 3 describes the elaborator's architecture and each component of the pipeline: parsing, type checking, evaluation, conversion checking, inductive type elaboration, and the incremental build and language server infrastructure. Section 4 presents measurements of correctness, batch elaboration performance, conversion checking speed, and the benefit of incrementality. Section 5 summarises the results and discusses future work.

Chapter 2

Preparation

2.1 Dependent type theory

We implement a dependent type theory in the tradition of Martin-Löf. The reader is assumed to be familiar with the simply typed lambda calculus; we focus on the features that distinguish our system.

2.1.1 Syntax

The core theory maintains a syntactic distinction between *types* and *terms* (Tarski-style universes). The grammar is:

$$\ell ::= n \mid 0 \mid \mathsf{succ}(\ell) \mid \max(\ell, \ell) \qquad \text{(universe levels)}$$

$$A, B ::= \mathsf{Type}_\ell \mid \Pi(x : A).B \mid \mathsf{El}(t) \qquad \text{(types)}$$

$$t, s ::= \mathsf{Type'}_\ell \mid x_i \mid c.\{\overline{\ell}\} \mid \lambda(x : A).t \mid t\ s \mid \Pi'(x : t).s \mid t.k \mid \mathsf{let}\, x : A := t\, \mathsf{in}\, s \qquad \text{(terms)}$$

Universe levels ℓ form an algebra of named variables n, zero, successor, and binary maximum. Types include universes Type_ℓ, dependent function types $\Pi(x : A).B$, and the decoding operation $\mathsf{El}(t)$ which converts a term-level code into a type. The term formers $\mathsf{Type'}_\ell$ and $\Pi'(x : t).s$ are codes for the corresponding type formers. Variables x_i are de Bruijn indices, $c.\{\overline{\ell}\}$ are global constants applied to universe level arguments, and $t.k$ is projection of the k-th field.

In Russell-style theories (as in Lean 4 itself), types and terms share a single syntactic sort: a Type expression may appear both as a type and as a term. In our Tarski-style formulation, every position in the syntax tree is unambiguously either a type or a term. The decoding operation El bridges the two: if $\Delta; \Gamma \vdash t : \mathsf{Type}_\ell$, then $\mathsf{El}(t)$ is a well-formed type.

Russell-style universes admit a simpler implementation — a single syntactic sort removes the need for `El` coercions and duplicated type/term formers. We choose Tarski style despite this cost because it admits a cleaner categorical semantics: Type_ℓ is an object and El is a morphism, corresponding directly to the structure of a universe in a category with families. The two styles are known to be equivalent up to translation, but the translation is non-trivial, and the metatheory of Tarski universes is easier to present and reason about formally.

2.1.2 Judgement forms

The theory has six judgement forms, parameterised by a global environment Δ and a local context Γ:

$\vdash \Delta\,\mathsf{sig}$	global environment well-formedness
$\Delta;\Gamma \vdash$	context well-formedness
$\Delta;\Gamma \vdash A\,\mathsf{type}$	type well-formedness
$\Delta;\Gamma \vdash t : A$	term typing
$\Delta;\Gamma \vdash A \equiv B\,\mathsf{type}$	judgemental type equality
$\Delta;\Gamma \vdash a \equiv b : A$	judgemental term equality

The global environment Δ maps constant names to their definitions and types. The local context Γ is a telescope of typed bindings.

2.1.3 Type formation rules

$$\frac{\Delta;\Gamma \vdash}{\Delta;\Gamma \vdash \mathsf{Type}_\ell\,\mathsf{type}}\,\mathsf{U} \qquad \frac{\Delta;\Gamma \vdash t : \mathsf{Type}_\ell}{\Delta;\Gamma \vdash \mathsf{El}(t)\,\mathsf{type}}\,\mathsf{El} \qquad \frac{\Delta;\Gamma \vdash A\,\mathsf{type} \quad \Delta;\Gamma, x : A \vdash B\,\mathsf{type}}{\Delta;\Gamma \vdash \Pi(x : A).B\,\mathsf{type}}\,\Pi\text{-}\mathsf{F}$$

2.1.4 Typing rules

$$\frac{\Delta;\Gamma \vdash}{\Delta;\Gamma \vdash x_i : \Gamma(i)}\,\mathsf{Var} \qquad \frac{\Delta;\Gamma \vdash \quad \Delta(c) = A}{\Delta;\Gamma \vdash c.\{\overline{\ell}\} : A}\,\mathsf{Const}$$

$$\frac{\Delta;\Gamma \vdash A\,\mathsf{type} \quad \Delta;\Gamma, x : A \vdash b : B}{\Delta;\Gamma \vdash \lambda(x : A).b : \Pi(x : A).B}\,\Pi\text{-}\mathsf{I} \qquad \frac{\Delta;\Gamma \vdash f : \Pi(x : A).B \quad \Delta;\Gamma \vdash a : A}{\Delta;\Gamma \vdash f\,a : B[a/x]}\,\Pi\text{-}\mathsf{E}$$

$$\frac{\Delta;\Gamma \vdash a : A \quad \Delta;\Gamma, x : A \vdash b : B}{\Delta;\Gamma \vdash \mathsf{let}\,x : A := a\,\mathsf{in}\,b : B[a/x]}\,\mathsf{Let} \qquad \frac{\Delta;\Gamma \vdash t : A \quad \Delta;\Gamma \vdash A \equiv B\,\mathsf{type}}{\Delta;\Gamma \vdash t : B}\,\mathsf{Conv}$$

The conversion rule allows a term of type A to be given type B whenever A and B are definitionally equal.

2.1.5 Definitional equality

Definitional equality is the congruence closure of the following computation rules:

- β-reduction: $(\lambda(x : A).b)\,a \equiv b[a/x]$
- δ-reduction: $c.\{\overline{\ell}\} \equiv t$ when $\Delta(c) = t$
- ζ-reduction: $\mathsf{let}\,x : A := a\,\mathsf{in}\,b \equiv b[a/x]$
- η-conversion: $f \equiv \lambda(x : A).f\,x$ at function type
- ι-reduction: recursor applied to a constructor computes to the corresponding branch

The full relation includes congruence, symmetry, transitivity, and closure under the conversion rule for types.

2.1.6 Universe polymorphism

Definitions may be parameterised by universe level variables. For example:

```
def id.{u} (α : Type u) (x : α) : α := x
```

At each use site, levels are instantiated explicitly: `id.{0}` or `id.{v}`. There is no universe level inference.

2.1.7 Inductive types

The theory supports user-defined inductive families. An inductive declaration introduces a type, its constructors, and a *recursor*. We illustrate with `Vector`, a length-indexed list:

```
inductive Vector.{u} (α : Type u) : Nat → Type u where
  | nil : Vector α Nat.zero
  | cons (n : Nat) (head : α) (tail : Vector α n) : Vector α (Nat.succ n)
```

This declaration generates:

- The type `Vector.{u} : (α : Type u) → Nat → Type u`
- Constructors `Vector.nil.{u} : (α : Type u) → Vector α Nat.zero` and `Vector.cons.{u} : (α : Type u) → (n : Nat) → α → Vector α n → Vector α (Nat.succ n)`
- A recursor `Vector.rec.{v, u}` that eliminates a `Vector` by providing a case for `nil` and a case for `cons`:

```
Vector.rec.{v, u} :
  (α : Type u) →
  (C : (n : Nat) → Vector α n → Type v) →
  C Nat.zero (Vector.nil α) →
  ((n : Nat) → (head : α) → (tail : Vector α n) → C n tail → C (Nat.succ n)
(Vector.cons α n head tail)) →
  (n : Nat) → (xs : Vector α n) → C n xs
```

All computation on inductives proceeds through the recursor. There is no primitive pattern matching. The computational rule (*iota-reduction*) fires when the recursor is applied to a constructor — for example, applying `Vector.rec` to `Vector.cons` reduces to the `cons` branch with the fields and recursive result as arguments.

As an example, `Vector.map` applies a function to each element:

```
def Vector.map.{u, v} (α : Type u) (β : Type v) (n : Nat) (f : α → β)
    : Vector.{u} α n → Vector.{v} β n :=
  Vector.rec.{v, u} α (fun k _ => Vector.{v} β k)
    (Vector.nil.{v} β)
    (fun m h _ => Vector.cons.{v} β m (f h))
    n
```

Structures (single-constructor inductives) additionally support *projections* $t.k$, which extract the k-th field from a constructor application.

2.2 Elaboration

The typing rules above are declarative: they specify *what* is well-typed, but not *how* to check it. Elaboration is the algorithmic process that implements these rules. We describe the three components of our elaboration pipeline: bidirectional type checking directs the flow of type information, normalisation by evaluation decides definitional equality efficiently, and conversion checking controls how aggressively definitions are unfolded.

2.2.1 Bidirectional type checking

Bidirectional type checking [8] splits type checking into two mutually recursive judgements:

- **Checking** $\Gamma \vdash e \leq A$: verify that e has type A, where A is already known.
- **Inference** $\Gamma \vdash e \Rightarrow A$: compute the type A of e.

Type information flows *downwards* in checking mode and *upwards* in inference mode. The bidirectional presentation partitions each rule by mode:

$$\frac{}{\Gamma \vdash x_i \Rightarrow \Gamma(i)}\,\mathsf{Var} \qquad \frac{\Gamma \vdash e \leq A}{\Gamma \vdash (e : A) \Rightarrow A}\,\mathsf{Anno}$$

$$\frac{\Gamma, x : A \vdash b \leq B}{\Gamma \vdash \lambda x.b \leq \Pi(x : A).B}\,\Pi\text{-}\mathsf{I} \qquad \frac{\Gamma \vdash f \Rightarrow \Pi(x : A).B \quad \Gamma \vdash a \leq A}{\Gamma \vdash f\, a \Rightarrow B[a/x]}\,\Pi\text{-}\mathsf{E}$$

$$\frac{\Gamma \vdash e \Rightarrow B \quad \Gamma \vdash A \equiv B\,\mathsf{type}}{\Gamma \vdash e \leq A}\,\mathsf{Sub}$$

Variables and applications are inferred; lambdas are checked against a known Π type. The subsumption rule Sub bridges the modes: an inferred term may be used in a checking position if its inferred type is convertible with the expected type. Each term form has a unique applicable rule in each mode, so the checker is syntax-directed and does not backtrack.

This design avoids metavariables entirely. Systems with implicit arguments (Lean, Agda, Rocq) use metavariables and pattern unification to fill in unsynthesisable information; our system instead requires the programmer to provide all type information explicitly. This keeps the elaborator simple and — crucially for incrementality — means that elaboration of one definition cannot produce side effects (solved metavariables) that affect other definitions.

2.2.2 Normalisation by evaluation

The Sub rule requires deciding definitional equality, which in turn requires reducing terms. *Normalisation by evaluation* (NbE) [9] evaluates syntax into a *semantic domain* of values, where definitional equality can be checked by structural comparison.

The semantic domain consists of *values* in weak head normal form. Closures — a term paired with its environment — stand for unevaluated function bodies. Neutral terms — a variable or constant head with a spine of eliminators — represent computations blocked on an unknown.

The syntax uses de Bruijn *indices* (counting from the innermost binder), while values use de Bruijn *levels* (counting from the outermost binder). This avoids the need to shift indices during substitution. *Evaluation* interprets syntax in an environment of values: variables are looked up, lambdas become closures, and applications perform beta-reduction or extend neutral spines. *Quotation* converts values back to syntax by applying closures to fresh variables and converting levels to indices.

A constant $c.\{\overline{\ell}\}$ is not unfolded during initial evaluation — it is left as a neutral. Its definition body is only retrieved and evaluated on demand, during weak head normalisation. This is significant for incrementality: the elaborator only creates a dependency on c's definition body if conversion checking actually needs to unfold it.

2.2.3 Glued evaluation

Unfolding every definition to its normal form is wasteful: most conversion checks succeed or fail long before full normalisation. *Glued evaluation* [10] addresses this by pairing each defined constant with two representations: a *folded* form (the neutral $c.\{\overline{\ell}\}$) and an *unfolded* form (the definition body, available lazily). A glued value $\mathsf{glued}(c.\{\overline{\ell}\}, t)$ carries both.

Conversion checking first compares the folded (head) forms. If both sides have the same head constant, they may be equal without unfolding. Only when the heads differ does the checker force the unfolded form. This reduces the amount of computation and — for incrementality — reduces the number of definition bodies the elaborator depends on.

2.2.4 Conversion checking

With these components in place, conversion checking compares two values structurally:

- If both sides are neutral with the same head, compare their spines.
- If both sides are lambdas, apply both to a fresh variable and compare the bodies.
- If both sides are glued with the same head constant, compare without unfolding.
- If the heads differ, unfold (delta-reduce) one or both sides and continue.

Following [10], we implement *approximate* conversion checking with three modes:

- **Rigid**: only unfold when the heads are rigid (constructors or distinct constants). This avoids unnecessary unfolding when variables block comparison.
- **Flex**: unfold defined constants but stop at variables. Used when rigid comparison fails.
- **Full**: unfold everything. A last resort when approximate methods are inconclusive.

The checker tries rigid first, then flex, then full. Each mode that avoids unfolding a definition body avoids creating a dependency on that body in the build system — so approximate conversion checking directly improves incrementality by narrowing the dependency graph.

2.2.5 The cost of elaboration

The dominant cost in elaboration is conversion checking. Type checking a single definition may trigger many conversion checks, each of which may unfold and normalise arbitrarily large terms. In a batch elaborator, this cost is paid once. But in an interactive setting, where a user edits one definition and expects rapid feedback, the question is: which of these conversion checks must be repeated?

A naive incremental system that invalidates everything after the edit point (as existing proof assistants do) repeats all of them. A query-based system can do better: it tracks which definitions each conversion check actually unfolded, and only repeats those checks whose dependencies have changed.

2.3 Incremental computation

2.3.1 Existing approaches

Existing proof assistants offer limited incrementality. Lean (since version 4.8) saves snapshots of elaboration state and resumes from the most recent valid checkpoint before an edit. Agda's

`--caching` flag reuses results for the unchanged prefix of declarations in interactive mode. coq-lsp re-checks from the first modified sentence onward. All three treat the file as a sequence and reprocess a suffix: if definition 5 of 200 changes, definitions 6 through 200 are re-checked, regardless of whether they depend on definition 5.

For conventional languages, query-based incrementality is well-established. Rust-analyzer uses Salsa [5], a framework that tracks dependencies between queries and uses early cutoff to avoid recomputation when results are unchanged. The key property that makes this work for Rust is the clean separation between signatures and bodies: changing a function body without changing its type cannot affect downstream type checking, so downstream queries are never invalidated.

Fredriksson's *sixty* [6] is the only prior work to apply query-based incrementality to a dependently typed language. It is built on Rock, a Haskell library inspired by the same "Build Systems à la Carte" framework. Sixty demonstrates that the approach is viable, but does not formalise the underlying build system or verify that incremental results agree with batch elaboration.

2.3.2 Why dependent types require dynamic dependencies

In a conventional compiler, the dependency graph between definitions is static: it is determined by imports and name resolution, and can be computed before type checking begins. A build system like Make, which requires dependencies to be declared upfront, suffices.

In a dependently typed elaborator, the dependency graph is not known upfront. Conversion checking may or may not need to unfold a given definition body, depending on the specific terms being compared — which in turn depend on the types of other definitions, which may themselves require conversion checking. Dependencies are discovered *during* elaboration.

This rules out build systems with static dependencies (Make, Bazel). What is needed is a system that supports *dynamic dependencies* — dependencies discovered during task execution — and *early cutoff* — skipping dependents when a recomputed result is unchanged. Early cutoff is especially important because definition bodies change more often than their types: without it, any body change would cascade through every downstream definition that *might* unfold it, even if the type is unaffected.

2.3.3 Build systems à la carte

Mokhov et al. [7] observe that build systems solve the problem of bringing outputs up to date with respect to changed inputs. They introduce a framework that captures Make, Shake, Bazel, and others as instances of a single polymorphic type.

The central abstraction is the *task*, which describes how to compute a value from its dependencies. A task is polymorphic in an effect f:

$$\mathsf{Task}\, c\, k\, v = \forall f.\, [c\, f] \Rightarrow (k \to f\, v) \to f\, v$$

A task receives a *fetch* callback that retrieves the value of any key, and produces its own value in the effect f, chosen by the build system. The constraint c on f determines what the task can do with the results of its fetches: if $c = \mathsf{Applicative}$, dependencies are static and known upfront; if $c = \mathsf{Monad}$, the task may inspect the result of one fetch to decide what else to fetch, giving dynamic dependencies.

A collection of tasks $\mathsf{Tasks}\, c\, k\, v = k \to \mathsf{Maybe}\, (\mathsf{Task}\, c\, k\, v)$ assigns a task to each non-input key. The paper decomposes build systems along two axes: the *scheduler* (topological, restarting, or

suspending) determines the order of task execution, while the *rebuilder* (dirty bits, verifying traces, or constructive traces) determines whether a key needs recomputation:

	Topological	Restarting	Suspending
Dirty bit	Make	Excel	-
Verifying traces	Ninja	-	Shake
Constructive traces	CloudBuild	Bazel	-
Deep constr. traces	Buck	-	Nix

Dependent type elaboration requires monadic tasks (dynamic dependencies) and verifying traces (early cutoff). This places us at the Shake cell of the design space — not by arbitrary choice, but because the properties of elaboration demand it.

The polymorphism of the `Task` type is what makes correctness modular. The elaborator defines tasks without knowing which build system will execute them; the build system executes tasks without knowing what they compute. Correctness decomposes into two independent obligations: (1) each task computes the right result given correct fetches, and (2) the build system calls tasks with valid inputs and caches correctly. This is analogous to the separation between Lean's elaborator and kernel: a small trusted component (the build system) guarantees a global property (incrementality is correct), independent of the complexity of the untrusted component (the elaborator).

2.3.4 Our formulation

We formalise the framework in Lean 4, making several refinements to the paper's Haskell presentation.

2.3.4.1 Separating inputs from queries

The paper uses a single key type `k` and distinguishes inputs from computed values by whether `Tasks` returns `Nothing`. We instead separate them at the type level: input keys `I` with values `V : I → Type`, and query keys `Q` with results `R : Q → Type`. A task can read inputs via `input i` and fetch query results via `fetch q`, as distinct operations:

```
def Task (α : Type) : Type 1 :=
  ∀ (f : Type → Type) [c f], (∀ i, f (V i)) → (∀ q, f (R q)) → f α
```

Tasks are now total — every query has a task, and `Maybe` is eliminated. The dependent types `V : I → Type` and `R : Q → Type` allow different queries to have different result types, which is essential for a type checker where "what is the type of x?" and "what are the declarations in file p?" return values of different types.

2.3.4.2 Well-founded termination

In order to properly define which sets of tasks terminate, we require that the dependency relation on queries be well founded. This relation is dependent on the set of inputs: given any set of inputs, the relation must be well founded.

The parameters are bundled into a configuration record:

```
structure BuildConfig : Type 1 where
  I : Type
  V : I → Type
  Q : Type
```

```
  R : Q → Type
  rel : (∀ i, V i) → Q → Q → Prop
  wf : ∀ ι, WellFounded (rel ι)
```

The relation `rel` is indexed by the input state `ι`: different source files can induce different dependency orders among queries. The task type carries the current input state ι_0 and origin query q_0, and the fetch callback requires a proof that the fetched query `q` precedes q_0:

```
def Task (α : Type) : Type 1 :=
  ∀ (f : Type → Type) [c f],
    (∀ i, f (𝒞.V i)) →
    (∀ q, 𝒞.rel ι₀ q q₀ → f (𝒞.R q)) →
    f α
```

This makes busy evaluation (the naive strategy that always recomputes) provably terminating by well-founded recursion, eliminating the need for runtime cycle detection.

2.3.4.3 Build system structure

A build system is a structure with private state `σ`, an initialiser, and a build function:

```
structure Build (J : Type) [Input 𝒞 J] : Type 1 where
  σ : Type
  init : J → σ
  inputs : σ → ∀ i, 𝒞.V i
  set : ∀ i, 𝒞.V i → StateM σ Unit
  build : Tasks c 𝒞 → ∀ q, StateM σ (𝒞.R q)
```

The `inputs` field extracts the current input state from the build state, while `set` updates an input value. The separation of inputs from queries enforces by construction that a build system cannot modify its own computed results — only inputs can be set externally.

2.3.5 Correctness

The goal of a correct build system is to produce the same result as naive batch evaluation. We define `compute`, the batch evaluator, by well-founded recursion: it runs each task in the identity monad, recursively computing every dependency from scratch.

```
def compute (tasks : Tasks c 𝒞) (ι : ∀ i, 𝒞.V i) (q : 𝒞.Q) : 𝒞.R q :=
  tasks q Id ι (fun q' _ => compute tasks ι q')
termination_by 𝒞.wf.wrap q
```

A verified build system is then a `Build` whose `build` function returns not just a result, but a *proof* that the result equals `compute`:

```
structure Build (c) (𝒞 : BuildConfig) (J : Type) [Input 𝒞 J] : Type 1 where
  σ : Type
  init : J → σ
  inputs : σ → ∀ i, 𝒞.V i
  set : ∀ i, 𝒞.V i → StateM σ Unit
  build :
    (tasks : Tasks c 𝒞) → (q : 𝒞.Q) → (store : σ) →
      { r : 𝒞.R q // r = compute tasks (inputs store) q } × σ
```

The return type `{ r // r = compute tasks (inputs store) q }` is a dependent pair: a value `r` together with a proof that `r` agrees with the batch evaluator. Any inhabitant of `Build` is correct by construction.

2.3.5.1 Busy: the batch baseline

The simplest correct build system is *Busy*, which calls `compute` directly. Its correctness proof is `rfl` — it *is* `compute`:

```
def Busy : Build c 𝒞 J where
  build tasks q store :=
    ((compute tasks (Input.get store) q, rfl), store)
```

2.3.5.2 LessBusy: memoisation

LessBusy memoises intermediate results in a cache, avoiding redundant recomputation within a single build. Each cache entry is a value paired with its correctness proof. LessBusy is proved correct using a *free theorem* (see below): if the memoised values are correct, then the overall build is correct.

2.3.5.3 Shake: incremental rebuilding

Shake extends memoisation across builds by storing fingerprints of each query's dependencies. On a subsequent build, Shake checks whether the recorded fingerprints still match the current inputs and dependency results. If they do, the cached value is reused without re-executing the task.

The correctness invariant is carried inside each cached entry:

```
structure Memo (q : 𝒞.Q) where
  value : 𝒞.R q
  inputDeps : List ((i : 𝒞.I) × H)
  deps : List (Σ' (q' : 𝒞.Q) (_ : 𝒞.rel q' q), H)
  invariant :
    ∀ (ι : ∀ i, 𝒞.V i),
      (∀ p ∈ inputDeps, hI p.1 (ι p.1) = p.2) →
      (∀ p ∈ deps, hR p.1 (compute tasks ι p.1) = p.2.2) →
      value = compute tasks ι q
```

The `invariant` field states: if every recorded input fingerprint matches the current input, and every recorded dependency fingerprint matches the current result of that dependency, then the cached `value` equals batch evaluation. Shake takes injective fingerprint functions `hI : ∀ i, 𝒞.V i ↪ H` and `hR : ∀ q, 𝒞.R q ↪ H` as parameters, so fingerprint equality implies value equality.

All three build systems — Busy, LessBusy, and Shake — inhabit the same `Build` type. Their correctness is established at construction time. The elaborator's tasks are a single `Tasks` value, independent of which `Build` executes them, so incremental elaboration is provably equivalent to batch.

2.3.5.4 The free theorem

The correctness proofs of LessBusy and Shake rely on a *relational parametricity* result for tasks. Because a `Task` is polymorphic in its effect f, running it in two different monads with pointwise-related inputs and fetches produces related results:

```
axiom Task.freeTheorem :
    (hι : ∀ i, F.rel Eq (ι₁ i) (ι₂ i)) →
    (hfetch : ∀ q hq, F.rel Eq (fetch₁ q hq) (fetch₂ q hq)) →
    F.rel Eq (t κ₁ ι₁ fetch₁) (t κ₂ ι₂ fetch₂)
```

This is admitted as an axiom: relational parametricity is a metatheoretic property of System F that cannot be proved internally in Lean. It is the standard justification for "theorems for free" results. The axiom is used once, to bridge the gap between a memoising run (in `StateM`

`Cache`) and the batch evaluator (in `Id`): if the cache returns correct values for every fetch, then the overall task result is correct.

2.4 Related work

2.4.1 Efficient elaboration

Kovacs's *smalltt* [10] demonstrates that high-performance elaboration is achievable through careful attention to evaluation strategies. It combines higher-order abstract syntax (HOAS), glued evaluation, and approximate conversion checking to achieve type-checking speeds that significantly outperform production systems on benchmarks. Several of these techniques — particularly approximate conversion checking with rigid/flex/full modes and glued evaluation — directly influenced the design of our elaborator.

2.4.2 Lean 4

Lean 4 [1] is the primary reference for the surface language and inductive type machinery. Our elaborator supports the same declaration forms (`def`, `inductive`, `structure`, `axiom`) and the same recursor-based elimination. The core theory departs from Lean's kernel in using Tarski-style rather than Russell-style universes, and in omitting metavariables and implicit arguments entirely.

2.5 Starting point

Prior to starting the project, I had experience implementing type checkers for simply typed languages, but had not implemented a dependently typed elaborator or a build system. I was already familiar with type theory, and had experience using Lean 4 as a proof assistant, but had not used it as a general-purpose programming language.

No implementation code was written before the project started. The project was initially planned in Rust, but an early switch was made to OCaml, and then to Lean 4. These transitions occurred during the project and are discussed in the Implementation chapter. No existing codebases were used as a basis; several projects were consulted as references, including smalltt, sixty, and Lean 4′s source.

Chapter 3

Implementation

This chapter describes the implementation of the elaborator. We follow the pipeline from source text to elaborated output: parsing, bidirectional type checking, normalisation by evaluation, conversion checking, inductive type elaboration, and the incremental build architecture that ties them together.

The implementation is written in Lean 4. Throughout this chapter, code snippets are excerpts from the implementation.

3.1 Architecture

We structure the elaborator as a collection of cooperating components. Source text is parsed by a hand-rolled monadic Pratt parser (Section) into a concrete syntax tree and a desugared abstract syntax tree. The AST is elaborated by a bidirectional type checker (Section) that uses normalisation by evaluation (Section) to reduce terms and a conversion checker (Section) to decide definitional equality. Inductive declarations (Section) are elaborated into types, constructors, and recursors. All of this is decomposed into fine-grained queries memoised by a Shake-based build system (Section), and diagnostics and hover information are served to a VSCode extension via the Language Server Protocol (Section).

The core of the elaborator is the `ElabM` monad:

```
abbrev ElabM :=
  Task Monad config ι₀ q₀
  |> StateT ElabState
  |> ReaderT ElabContext
```

Here `config : BuildConfig` bundles the input and query types, while `ι₀` and `q₀` are the current input state and origin query carried along for the well-founded dependency relation.

Each transformer serves a distinct purpose:

- `Task` is the incremental query monad. Fetches through `Task.fetch` register dependencies in the Shake graph.
- `StateT ElabState` threads mutable elaboration state: the local environment of constants elaborated so far, a cache of fetched constants, and accumulated diagnostics and hover information.

- `ReaderT ElabContext` carries read-only context: the file path, the current declaration name, the universe parameter list of the current definition, and a path into the AST for diagnostic positions.

An earlier iteration of the design used `WriterT ElabInfo` instead of threading diagnostics through `ElabState`. Microbenchmarks showed this introduced significant overhead: every monadic `>>=` invoked the monoid append on the `ElabInfo` tuple, even when nothing was emitted. Replacing `WriterT` with a push-only interface over `StateT` arrays produced a measurable speedup on repeated-conversion benchmarks.

The components are connected by query dependencies, not by direct function calls. The parser's output is the `Val` of a `Key.cst` query; the AST is the `Val` of a `Key.ast` query; each declaration's elaborated form is the `Val` of a `Key.elabDecl` query, which depends on `Key.constant` queries for every referenced name. This structure makes the system incremental: when an input changes, the build system walks the dependency graph backward and invalidates exactly the queries that transitively depended on it.

3.2 Parsing

The parser is a hand-rolled recursive descent Pratt parser, implemented as monadic combinators over `StateT State (Except ParseError)`. Parsing proceeds in two stages: the source text is first parsed to a concrete syntax tree (`Cst`) that retains trivia and exact token widths, and the CST is then desugared to an abstract syntax tree (`Ast`) that strips trivia and expands surface syntax.

The parser itself is unremarkable — every language needs one, and Pratt parsing is a well-understood technique for operator precedence. The interesting design choice is the representation of the CST.

3.2.1 Green trees

The CST follows the *green tree* pattern (used by Roslyn and rust-analyzer). Nodes and tokens are tagged by `SyntaxNodeKind`, and positions are not stored explicitly:

```
inductive Cst : Type
  | token (kind : SyntaxNodeKind) (val : String)
  | node (kind : SyntaxNodeKind) (children : Array Cst)
with
  @[computed_field]
  width : Cst → Nat
    | .token _ val => val.length
    | .node _ children => children.map width |>.sum
```

Each node carries only its kind and children; its *width* is a computed field summing the widths of its descendants. A source position is then reconstructed by walking from the root and summing the widths of preceding children. Because nodes do not store absolute positions, two identical subtrees are structurally equal regardless of where they appear in the file — the CST is `Hashable`, which lets the build system short-circuit recomputation when a file is edited in a way that does not affect a given subtree.

This matters for incrementality. The elaboration pipeline works entirely with AST *paths* (lists of child indices from the root), not source positions. Diagnostics and hover information are keyed by path, so the elaborated result — the `Constant`, the `ElabInfo` — is independent of where

the definition sits in the file. Source positions are only reconstructed on demand, by summing widths from the root, when the language server needs to report to the editor.

Because nodes do not store absolute positions, adding whitespace before a definition changes the raw file content (invalidating `Key.cst`) but leaves the green tree subtree for that definition structurally identical — same kind, same children, same widths. The AST and all downstream elaboration queries hash the same, so they are not recomputed. Without green trees, any whitespace edit would shift absolute positions throughout the file, changing every diagnostic and hover span, and invalidating every query that produced them.

3.2.2 Language server integration

Alongside the CST, parsing produces a `SourceMap` that bidirectionally maps paths in the CST to paths in the AST. A *path* is a list of child indices from the root, so the root of either tree has path `[]`, its first child has path `[0]`, and so on. The bidirectional map is:

```
structure SourceMap where
  cstToAst : HashMap Path Path
  astToCst : HashMap Path Path
```

The map is populated during desugaring: as each AST node is constructed from a CST node, an entry is added to both tables. The language server uses this in both directions:

- *Hover*: given a cursor position, the CST is walked to find the path of the token under the cursor, then `cstToAst` gives the AST path. The elaborator's hover information is keyed by AST path, so the correct hover content is retrieved directly.
- *Diagnostics*: the elaborator records diagnostics at AST paths. `astToCst` maps these back to CST paths, which are then converted to source spans by summing widths.

Without the source map, a position-based language server would need to traverse the entire AST comparing source ranges at every node. The path-based lookup is $O(\text{depth})$ on the tree rather than $O(\text{size})$.

3.2.3 Error recovery

The parser uses lightweight error recovery: on encountering a parse error, a diagnostic is emitted and the parser attempts to continue at the next top-level declaration boundary (`def`, `inductive`, `axiom`, etc.). This ensures that a single malformed declaration does not prevent the rest of the file from being elaborated — the language server continues reporting errors throughout the file.

A separate converter from `Lean.Syntax` to `Ast` is available via the metaprogramming framework, for writing inline test cases in Lean source files. This is not used by the main pipeline.

3.3 Bidirectional type checking

Type checking is bidirectional. Instead of a single judgement $\Gamma \vdash t : A$ that both produces and checks types, the algorithm is split into two mutually recursive functions:

- `inferTm ctx ast : OptionT ElabM (Tm n × VTy n)` — given an AST, synthesise a core term and its type. Used when the expected type is not known or would be recomputed redundantly.
- `checkTm ctx expectedTy ast : ElabM (Tm n)` — given an AST and an expected type, produce a core term of that type. Used when the expected type is already available from the

surrounding context. Failure is handled internally by emitting a diagnostic and inserting a fresh axiom as a placeholder.

The split has three benefits. First, many terms can be checked without needing to synthesise their types: a lambda's parameter type is determined by the expected Pi type, so the parameter can be left unannotated. Second, the algorithm is syntax-directed — each AST constructor dispatches to a specific case, avoiding the need for backtracking or unification. Third, errors are pinpointed: a mismatch between the expected and inferred type is reported at the exact subterm, rather than propagating upward.

The mode-switching rules are standard:

- **Lambdas** are checked. In checking mode against expected type $\Pi(x : A).B$, the body is checked against B in the context extended with $x : A$. If the AST's parameter has a type annotation, it must match A (via conversion checking); otherwise A is taken from the expected type.
- **Applications** are inferred. The function f is inferred, yielding a type T. If T is not a Pi type after weak head normalisation, the elaborator fails with a type error. Otherwise $T = \Pi(x : A).B$ and the argument a is checked against A; the result has type $B[a/x]$.
- **Variables** are inferred by looking them up in the local context or the global environment.
- **Let-bindings** are inferred by first inferring the bound value's type, then checking the body in the extended context.
- **Universes** Type_ℓ are inferred to have type $\mathsf{Type}_{\ell+1}$.
- **Subsumption**: when an inferred term is used in a checking position, the inferred type is compared against the expected type by the conversion checker.

The core loop is `inferTm`/`checkTm`, mutually recursive with a helper `checkIdent` that handles variable lookup (checking the local context first, then the global environment). For example, the application case of `inferTm` is:

```
| .node `Term.app #[fn, arg] => do
    let (fnTm, fnTy) ← inferTm ctx fn
    let .pi _ dom ⟨env, codTm⟩ := fnTy
      | raiseError (.expectedFunctionType ctx.names (← fnTy.quote))
    let argTm ← checkTm ctx dom arg
    let argVal ← argTm.eval ctx.env
    let codVal ← codTm.eval (env.cons argVal)
    return (.app fnTm argTm, codVal)
```

The function is inferred, yielding a type `fnTy`. Because evaluation eagerly unfolds constants into glued values, and `inferTm` always returns a `VTy` produced by `Ty.eval`, `fnTy` is already in weak head normal form — no explicit `whnf` call is needed to expose the Pi structure. The argument is then checked against the domain, and the codomain is substituted via evaluation in the closure's extended environment, rather than by explicit syntactic substitution.

When checking or inference fails — due to a `sorry`, an unbound variable, or a type mismatch — the elaborator emits a diagnostic and inserts a fresh axiom as a placeholder at the expected type. This keeps the rest of the file type-checkable, so the language server can continue reporting errors throughout the file rather than stopping at the first mistake.

3.3.0.1 Comparison with unification-based elaboration

Pure type inference (synthesising types for every subterm) is incomplete for dependent type theories: without additional structure, variables like `fun x => x` have no principal type. Systems

with implicit arguments and metavariables fill this gap via unification, solving for the missing types during elaboration. This is powerful but introduces substantial complexity — pattern unification, occurs checks, postponed constraints, meta freezing — none of which appear in this implementation.

Bidirectional type checking [8] sidesteps this by requiring the user to annotate just enough types that checking is deterministic. In practice the annotation burden is small: top-level definitions have types (because they are either explicit or inferred from a body), lambdas under a Pi inherit their domain, and let-bindings propagate their inferred type to the body. The cases that require annotations — such as the binder type of a top-level lambda with no outer context — are exactly the cases where the user would need to provide the information anyway.

The approach taken here is syntax-directed with no metavariables and no backtracking. The trade-off is that the surface language is more verbose than Lean's or Agda's, but the elaborator is significantly simpler, which was a deliberate choice given the scope of this project.

3.4 Normalisation by evaluation

3.4.1 Semantic domain

The semantic domain separates *values* from *syntax*. Values (`VTm`, `VTy`) represent terms in weak head normal form, using de Bruijn *levels* rather than indices:

```
inductive VTm : Nat → Type
  | u'      : Universe → VTm n
  | neutral : Neutral n → VTm n
  | lam     : Name → VTy n → ClosTm n → VTm n
  | pi'     : Name → VTm n → ClosTm n → VTm n
  | glued   : Neutral n → Tm 0 → VTm n
```

A `neutral` is a head (variable or constant) applied to a spine of eliminators. A `lam` carries a closure — an environment `Env n m` paired with a body term `Tm (m + 1)` having one free variable. Beta-reduction evaluates the body in the environment extended with the argument. A `glued` value carries both a neutral (the folded form, for quotation) and the unevaluated definition body (for reduction).

The choice of de Bruijn levels for values and indices for syntax is deliberate. Under levels, a variable's name is absolute rather than relative, so weakening (embedding a value into a larger scope) is a runtime no-op: the level remains the same when additional variables are pushed onto the environment. Under indices, every weakening requires traversing the value to shift every variable. Since NbE weakens values frequently — closures are opened at fresh levels during quotation and conversion — this would be a significant cost. The scope index `n` in `VTm n` enforces well-scoping at the type level; weakening then becomes a proof obligation that `omega` discharges automatically, and the underlying data is reused via `unsafeCast`.

3.4.1.1 Choice of closure representation

Closures in NbE can be represented in two ways:

- **Higher-order abstract syntax (HOAS)**: a lambda carries a host-language function `VTm → VTm`. Beta-reduction is a function call; the host compiler's closure optimisations apply.

- **Defunctionalised closures**: a lambda carries the body as syntax paired with its captured environment. Beta-reduction re-interprets the body by evaluating it in the extended environment.

HOAS is significantly faster in practice: smalltt [10] reports a 2–3× speedup over defunctionalised closures on identical benchmarks, because GHC compiles the host function to efficient native code. However, HOAS is strictly positive only in languages that permit non-positive inductive types. Lean's kernel rejects `VTm → VTm` in the `lam` constructor, as it violates positivity and opens the door to non-termination via Curry's paradox.

We could bypass the check with `unsafe inductive`, as the normalisation benchmarks in Lean themselves do. This was considered, but rejected: marking `VTm` unsafe would prohibit proving any properties of the evaluator, which in turn would prohibit proving the correctness of conversion checking and elaboration. The defunctionalised representation keeps the evaluator within the kernel's logic.

3.4.2 Evaluation

Evaluation (`Tm.eval`) interprets syntax in an environment of values. The environment is indexed by the scope size, matching the scope index of the syntax being evaluated:

```
partial def Tm.eval {n c} : Tm c → SemM n c (VTm n)
  | .u' i         => return .u' i
  | .var i        => return (← read).get i
  | .const name us => do
      let some tm ← fetchDefinition name
        | return .neutral ⟨.const name us, .nil⟩
      let some info ← fetchConstantInfo name
        | return .neutral ⟨.const name us, .nil⟩
      return .glued ⟨.const name us, .nil⟩
        (tm.substLevels (info.univParams.zip us))
  | .lam x a body => return .lam x (← a.eval) ⟨← read, body⟩
  | .app fn arg   => do (← fn.eval).app (← arg.eval)
  | .pi' x a b    => return .pi' x (← a.eval) ⟨← read, b⟩
  | .proj i t     => do (← t.eval).proj i
  | .letE _ _ t b => do b.eval (.cons (← t.eval) (← read))
```

The cases are:

- **Variables** are looked up in the environment by de Bruijn index.
- **Constants** evaluate to *glued* values when a definition is available: the result carries both the folded neutral form and the unevaluated definition body (with universe parameters substituted). If the constant has no definition (e.g. an axiom or an opaque declaration), a plain neutral is returned.
- **Lambdas** create closures capturing the current environment and the unevaluated body. No reduction happens under the binder.
- **Applications** evaluate both sides, then dispatch on the function's head in `VTm.app`: beta-reduction if it is a lambda, extending the spine if it is neutral or glued.
- **Let-bindings** evaluate the bound term and extend the environment, then evaluate the body.

Weak head normalisation (`VTm.whnf`) unfolds further on demand: for a constant-headed neutral, it delta-reduces (fetches and evaluates the definition) and re-applies the accumulated spine. For a glued value, it forces the unfolded body. It also performs iota-reduction when a fully-applied recursor has a constructor as its major premise.

3.4.3 Quotation

Quotation (`VTm.quote`) converts values back to syntax. For closures, it applies the closure to a fresh variable at the current de Bruijn level and quotes the result, converting levels to indices during the traversal. For `VTm.glued` nodes, quotation uses the folded form (the neutral), producing compact output that preserves top-level definition names rather than inlining their bodies.

3.5 Conversion checking

Conversion checking decides definitional equality of two values. The naive algorithm is: reduce both sides to normal form, then compare structurally. This is correct but slow, because it forces all reducible subterms regardless of whether comparison actually needs them.

The approximate algorithm used here, inspired by Kovacs's smalltt [10], bounds the amount of speculative work by a three-state mode parameter:

```
inductive ConvState where
  | rigid
  | flex
  | full
```

The three modes are:

- **Rigid**: the default mode, used at the top level and under canonical formers (lambdas, Pi types, constructors). When two neutral or glued values have the same defined head, spines are compared speculatively in flex mode. If the flex comparison succeeds, no unfolding was needed — the two terms are structurally equal at the folded level. If it fails, both sides are unfolded to their definitions and compared in full mode.
- **Flex**: entered only from rigid's speculative spine check. No definitions are unfolded; no fallback is attempted. If heads agree, the spines are compared elementwise, still in flex mode. Any mismatch causes immediate failure, returning control to the rigid caller, which then commits to unfolding.
- **Full**: entered from rigid after a flex failure. All definitions are unfolded immediately on encounter. Once in full mode, all recursive calls remain in full.

This ensures at most one backtrack per subterm: rigid attempts a cheap flex check, and on failure commits to the expensive full check. Because flex never unfolds, the cost of a failed speculation is bounded by the size of the matching prefix of the two spines.

To see the benefit, consider comparing `f (g (h x))` with `f (g (h x))` where `f`, `g`, `h` are top-level definitions. The naive algorithm unfolds `f`, `g`, `h` on both sides and compares the unfolded bodies — potentially an exponentially large computation if the definitions are deeply nested. The approximate algorithm observes that the two terms have the same folded structure: same head `f`, same spine. The flex comparison descends into the argument, observes the same head `g`, and so on, bottoming out at `x` with no unfolding at all. The check runs in time proportional to the syntactic size of the folded term, not the unfolded term.

For eta-conversion, two cases are handled: function eta, where $f \equiv \lambda x.\, f\, x$ is decided by opening both sides at a fresh variable and comparing the bodies; and structure eta, where $c(r_1, ..., r_k) \equiv s$ (with c a constructor of a single-constructor inductive) is decided by comparing each projection $s.i$ with the corresponding field r_i.

3.5.0.1 Comparison with naive reduction

A naive conversion checker would reduce both sides to normal form and compare. This has two problems. First, normal forms can be exponentially larger than the original terms: consider a definition `def big := ...` that unfolds to a deeply nested expression; comparing `big` with itself would first materialise the huge normal form, then walk it structurally, even though the syntactic equality `big = big` is immediately decidable. Second, the comparison walks the entire normal form even when a mismatch is present near the surface, wasting work.

The rigid/flex/full algorithm addresses both problems. Flex mode exploits the observation that most conversion checks succeed on identical or structurally similar terms: a flex comparison succeeds without any unfolding, running in time proportional to the syntactic size. When flex fails, the algorithm commits to full unfolding, but only for the specific subterm where speculation failed — not for the whole expression. The bounded backtracking ensures that the asymptotic cost is at most the minimum of the two strategies: the flex cost if speculation succeeds, or the flex cost plus the full cost if it fails.

An alternative design, used in some systems, is *on-the-fly* reduction: weakly head normalise each side just enough to decide equality at each level, and recurse under binders. This avoids eagerly computing full normal forms, but without the rigid/flex distinction it still unfolds aggressively whenever the heads do not syntactically match. The speculative flex check avoids this unfolding in the common case where heads are the same defined constant.

3.6 Inductive types

Inductive declarations are the means by which the user extends the theory with new data types. A declaration

```
inductive I.{u} (P : Type u) : Nat → Type u where
  | c₁ : ... → I P n₁
  | c₂ : ... → I P n₂
```

introduces the *type former* `I`, a family of *constructors* `c_i`, and a *recursor* `I.rec`. Each constructor is a term that builds values of `I` at specific indices; the recursor is the unique non-dependent eliminator that computes on each constructor.

The elaboration process proceeds in several phases:

1. **Elaborate the type.** Parameters and indices are elaborated in order. The result type must be a universe $\texttt{Type}_u$. Note that the distinction between parameters and indices is syntactic: parameters appear before the `:`, indices after.
2. **Register the inductive as opaque.** Before elaborating constructors, the inductive is added to the global environment as an opaque declaration with just its type. This lets constructors refer to the inductive itself during their own elaboration (via the constant name), without creating a circular dependency.
3. **Elaborate each constructor.** Each constructor's fields are elaborated in a context that binds a variable `I` representing the inductive (not the actual constant, which is not yet fully registered). The constructor's result type must be `I` applied to the inductive's parameters *verbatim* — the same de Bruijn indices as the parameter telescope — followed by chosen indices. This constraint (`ctorParamMismatch` on mismatch) simplifies the recursor generation, which relies on parameters being invariant across constructors.

4. **Check strict positivity.** In each field's type, the inductive may appear only in *strictly positive* positions. Non-positive occurrences are rejected as `nonPositiveOccurrence` errors. The positivity analysis permits *nested* recursion: a field may be a function type whose result mentions the inductive, provided the inductive does not appear in any argument type of that function.
5. **Check universe consistency.** Each field's universe level must be bounded by the inductive's result universe, rejecting `fieldUniverseTooLarge` otherwise. This ensures the inductive type lives in a universe large enough to contain all its fields.
6. **Generate the recursor type.** The recursor takes the parameters, a *motive* $C : \mathsf{indices} \to I\ \mathsf{parameters}\ \mathsf{indices} \to \mathsf{Type}_v$, one *minor premise* per constructor (which produces C applied to the constructor), the indices, and finally the major premise of type `I`. The motive's universe v becomes an additional universe parameter of the recursor (see below).
7. **Generate the recursor rules.** One rule per constructor: when `I.rec` is applied to `c_k args`, it reduces to the k-th minor premise applied to `args`. Recursive fields (where the inductive appears in the field's type) additionally contribute *induction hypotheses* computed by recursive calls to `I.rec`.
8. **Replace the inductive entry.** Finally, the opaque placeholder registered in step 2 is replaced with the full inductive declaration, including the list of constructor names.

For example, the recursor for `Nat`:

$$\mathsf{Nat.rec}.\{u\} : (C : \mathsf{Nat} \to \mathsf{Type}_u) \to C\,\mathsf{zero} \to ((n : \mathsf{Nat}) \to C\,n \to C\,(\mathsf{succ}\,n)) \to (n : \mathsf{Nat}) \to C\,n$$

with the iota-reduction rules:

$$\mathsf{Nat.rec}\,C\,z\,s\,\mathsf{zero} \rightsquigarrow z$$
$$\mathsf{Nat.rec}\,C\,z\,s\,(\mathsf{succ}\,n) \rightsquigarrow s\,n\,(\mathsf{Nat.rec}\,C\,z\,s\,n)$$

Structures (single-constructor inductives) additionally generate *projection* functions `I.f_k`, one per field, that extract the k-th field from a constructor application. The conversion checker includes a structural eta-expansion rule for structures: `c (s.f_1) ... (s.f_k)` is identified with `s`.

3.6.0.1 Universe handling

Universe polymorphism makes recursor generation subtle: the motive's universe may differ from the inductive's universe. For example, elimination from `Nat` into `Type 0` gives ordinary recursion, while elimination into `Type (u+1)` gives large elimination.

The elaborator handles this by generating a fresh universe parameter `motiveUnivName`, distinct from all universe parameters of the inductive itself, and prepending it to the recursor's universe parameter list. So for a monomorphic inductive like `Nat` (`univParams = []`), the recursor has `recUnivParams = [motiveUnivName]`. For `List.{u}` (`univParams = [u]`), the recursor has `recUnivParams = [motiveUnivName, u]`. At each use site, `Nat.rec.{v}` instantiates the motive's universe to `v`.

The freshness is ensured by `Universe.freshName`, which picks a name not in the inductive's universe parameter list. The convention is that the motive's universe is the *first* universe parameter of the recursor.

3.6.0.2 Recursors rather than pattern matching

Modern proof assistants like Lean, Agda, and Coq compile user-facing pattern matching to recursor applications internally. Pattern matching is more convenient for users, but significantly

more complex to implement: case trees, catch-all patterns, dependent case analysis, and coverage checking all add machinery.

The core theory here uses recursors directly, as Coq did originally. This keeps the core simple and the trusted code base small: the elaborator need only check that constructors are well-typed and generate the recursor; no pattern-matching compiler is needed. The cost is ergonomics — stdlib definitions are written with explicit recursor calls rather than `match` — but the core theory is small enough that adding a pattern-matching frontend would be straightforward future work.

3.7 Incremental elaboration

Elaboration is decomposed into queries managed by a Shake-based build system. Each query is identified by a tag and parameters, and returns a value whose type is determined by the tag. The query types are defined as a pair of indexed inductive types. The actual `Key` type has 17 cases; the principal ones are:

```
inductive Key where
  | cst : FilePath → Key
  | ast : FilePath → Key
  | declarationIndex : FilePath → Key
  | elabCmdAt : FilePath → Nat → Key
  | elabDecl : FilePath → Name → Key
  | constant : FilePath → Name → Key
  | lookupInfo : FilePath → Name → Key
  -- ... others omitted

def Val : Key → Type
  | .cst _              => Cst × Array ParseError
  | .ast _              => Ast
  | .declarationIndex _ => HashMap Name Nat × Array Diagnostic
  | .elabCmdAt _ _      => Global × ElabInfo
  | .elabDecl _ _       => Option (Constant × Origin) × ElabInfo
  | .constant _ _       => Option (Constant × Origin)
  | .lookupInfo _ _     => ElabInfo
  -- ... others omitted
```

The dependent typing of `Val` is essential: a single query map cannot be used to store results of different types without either existential packing or a universal result type. Using a type-level function, each query's result type is precisely determined by its key.

The principal queries are:

- `declarationIndex`: given a file path, returns a map from declaration names to their indices within the file. This query depends only on the parsed AST, so it is recomputed only when the file is edited.
- `elabDecl`: elaborates a single declaration. It fetches the AST, looks up the declaration by name, and type-checks it. It depends on `constant` queries for every constant it references.
- `constant`: looks up the elaborated form of a named constant. This is the primary cross-declaration dependency.

When the elaborator encounters a reference to a constant, it calls `fetchConstant`, which issues a `Key.constant` query through the build system. This registers a dependency: if the referenced constant changes, the current query is invalidated and recomputed on the next build.

Within a single elaboration run, an `entryCache` in `ElabState` memoises constant lookups to avoid repeated queries to the build system for the same name.

3.7.0.1 Granularity of queries

The choice of query granularity trades off overhead against incremental precision. Too coarse — e.g. one query per file — and any edit invalidates the whole file. Too fine — e.g. one query per expression — and the per-query overhead dominates, making the system slower than batch elaboration for typical edits.

The granularity here is per-declaration. A `Key.elabDecl` query elaborates one top-level `def`, `inductive`, or `structure`. When a declaration is edited, only that declaration's query is invalidated; dependent declarations are recomputed only if the edit changes the declaration's cached `Constant` (its elaborated type and body). This matches the granularity at which a user typically edits: they change one definition at a time, and expect the feedback loop to scale with the cost of that definition, not the whole file.

Finer granularity — for example, caching the elaborated form of individual subterms — would require treating each subterm as a named entity, which it is not in the source language. Implementing this would require either heuristic naming (hashing the subterm's syntax) or restructuring the core theory to make subterms addressable. Neither is compelling for the cost.

3.7.0.2 Dependently-typed query results

An alternative to the dependent type function `Val : Key → Type` is to use a single result type that subsumes all query results, e.g. a sum type `inductive Result | parsed : ... | elaborated : ... | ...` or an existential `Σ α, α`. Both have drawbacks:

- A sum type forces every query to return a tagged value and every consumer to pattern-match and handle an "impossible" case when the tag does not match. This is boilerplate that the dependent typing eliminates.
- An existential type erases the result type entirely, forcing `unsafeCast` or a proof-carrying wrapper at every fetch site. Neither integrates cleanly with the elaborator's other structures.

The dependent `Val` makes `fetch (Key.constant p n) : Task (Option Constant)` type-check without any tagging, and the compiler can specialise each call site to the specific result type. This is a natural use of Lean's dependent types that has no direct counterpart in Haskell's Rock or Salsa implementations, where singleton types or type families are needed to approximate the same pattern.

3.8 Language server

The elaborator doubles as a language server. During elaboration, hover information and diagnostics are collected in `ElabState` and returned alongside the elaboration result. A VSCode extension communicates with the elaborator via the Language Server Protocol, providing:

- **Diagnostics**: type errors, unbound variables, and universe mismatches, positioned at the relevant source location via path indices into the AST.
- **Hover information**: the type of the term under the cursor, or the full signature of a referenced constant.

Chapter 4

Evaluation

We evaluate the elaborator along three axes: correctness, performance, and incrementality.

4.1 Correctness

The primary correctness test is the standard library. The stdlib contains 2,200 lines of qdt code across 36 files, covering natural number arithmetic, propositional equality, well-founded recursion, sigma types, monadic abstractions, and the Ackermann function. Each definition is fully explicit — no implicit arguments or wildcards — and type-checked from scratch on every run.

Equality proofs in the stdlib serve as integration tests for the conversion checker: a proof `Eq.refl.{0} Nat 6` at type `Nat.add 2 4 = 6` succeeds only if the elaborator correctly reduces `Nat.add 2 4` to `6` via iota-reduction on the recursor.

4.2 Batch elaboration performance

We measure the time to elaborate the full stdlib from a cold start, with no cached results.

Cold-start elaboration of the full stdlib completes in approximately 438ms (median of three runs: 447ms, 408ms, 438ms).

4.3 Conversion checking performance

To stress-test the evaluator and conversion checker in isolation, we use Church-encoded benchmarks adapted from Kovacs's normalisation-bench [10]. These encode natural numbers and binary trees as lambda terms, then check definitional equality between two independently computed values of the same normal form.

The benchmarks are:

- **Nat n conv**: check that two Church numerals, both representing n, are definitionally equal. This forces n beta-reductions on each side.
- **Tree n conv**: check that two full binary trees of depth n (with $2^{n+1} - 1$ nodes) are definitionally equal.

Nat 10K conv (checking equality of two Church numerals representing 10,000) completes in approximately 136ms (median of three runs: 139ms, 134ms, 136ms). Larger benchmarks (Nat 100K and above) overflow the stack. This is a known limitation of the defunctionalised closure representation used in the evaluator — each beta-reduction is a recursive call, and Church numerals with n successors require n stack frames.

For comparison, smalltt handles Nat 5M conv in approximately 90ms and Nat 10M conv in 180ms, using HOAS closures compiled by GHC. The stack overflow in our implementation prevents direct comparison at these scales.

4.4 Incremental re-elaboration

The key benefit of the query-based architecture is that editing a single definition does not require re-checking the entire file. We measure the time to re-elaborate after modifying a definition, compared to batch re-elaboration.

TODO: table comparing incremental vs batch times for targeted edits.

4.5 Discussion

TODO

Chapter 5

Conclusion

5.1 Summary

We have presented an incremental, query-based elaborator for a dependently typed language, implemented in Lean 4. The elaborator supports dependent function types, a hierarchy of Tarski-style universes with universe polymorphism, and inductive types with recursors. Elaboration is decomposed into fine-grained queries managed by a Shake-based build system, so that editing a single definition recomputes only the queries that depend on it.

The build system framework is formalised in Lean 4, refining the "Build Systems à la Carte" framework with a type-level separation of inputs from queries and well-founded termination of the dependency relation. The elaborator itself uses normalisation by evaluation for efficient term reduction and an approximate conversion checker with rigid/flex/full modes to bound backtracking.

TODO: summarise evaluation results.

5.2 Future work

Several directions remain open:

- **Metavariables and implicit arguments.** The current system requires all terms to be fully explicit. Adding metavariables with pattern unification would make the surface language significantly more usable.
- **Parallelism.** The build system framework supports a base monad parameter that could be instantiated with IO for parallel query evaluation. This would allow independent queries to be elaborated concurrently.
- **Cancellation.** A language server should be able to cancel in-progress elaboration when the user edits the file again. The middleware framework could support this via an exception monad that preserves partial progress in the memo store.

Bibliography

[1] L. de Moura and S. Ullrich, "The Lean 4 Theorem Prover and Programming Language," in *Automated Deduction – CADE 28*, Springer, 2021, pp. 625–635. doi: 10.1007/978-3-030-79876-5_37.

[2] U. Norell, "Dependently Typed Programming in Agda," in *Advanced Functional Programming*, Springer, 2009, pp. 230–266. doi: 10.1007/978-3-642-04652-0_5.

[3] B. Barras *et al.*, "The Coq Proof Assistant Reference Manual," 1999.

[4] A. Kladov and contributors, "rust-analyzer." [Online]. Available: https://rust-analyzer.github.io/

[5] N. Matsakis and contributors, "Salsa: A generic framework for on-demand, incrementalized computation." [Online]. Available: https://github.com/salsa-rs/salsa

[6] O. Fredriksson, "sixty." [Online]. Available: https://github.com/ollef/sixty

[7] A. Mokhov, N. Mitchell, and S. Peyton Jones, "Build Systems à la Carte," *Proceedings of the ACM on Programming Languages*, vol. 2, no. ICFP, Sept. 2018, doi: 10.1145/3236774.

[8] J. Dunfield and N. Krishnaswami, "Bidirectional Typing," *ACM Computing Surveys*, vol. 54, no. 5, 2021, doi: 10.1145/3450952.

[9] A. Abel, "Normalization by Evaluation: Dependent Types and Impredicativity," *Lecture notes*. 2013.

[10] A. Kovács, "smalltt." [Online]. Available: https://github.com/AndrasKovacs/smalltt

Chapter 6

Project Proposal

Query-based Incremental Elaborator for a Dependently Typed Language

6.1 Introduction and Description

This project aims to implement an incremental, query-based elaborator for a dependently typed toy language, for efficient type checking in interactive development environments, such as proof assistants. Traditional type checkers for dependent types operate in batch mode, re-checking entire programs after each change. This project will investigate query-based compilation techniques, building on Olle Fredriksson's work and modern language servers, to enable efficient incremental type checking.

The core objective is to build an elaborator that can efficiently respond to local changes in source code by recomputing only the affected parts of the type checking process. This will be achieved using a query-based architecture where type checking operations are decomposed into memoisable queries with explicit dependencies. The implementation will use the Salsa framework, written in Rust, which provides the tools for incremental on-demand computation by automatically tracking dependencies and memoisation.

6.2 Starting Point

No prior work was done before October.

6.3 Substance and Structure

1. Core Language: a minimal dependently typed calculus with Pi, universes, and basic inductive types. The syntax will support function definitions, pattern matching, and type annotations.
2. Query-based elaborator: bidirectional type checking decomposed into queries (e.g., `infer_type`, `check_type`, `normalise_term`). Each query can be memoised, with proper dependency tracking to enable incremental recomputation.
3. Incremental infrastructure: use of the Salsa framework to handle source file changes, manage parse trees, and dependency graphs for queries. Detection of changes and lazy memoisation strategies. This is the most important component of the project.

4. Performance evaluation framework: benchmarking suite to measure incremental performance against batch re-elaboration, including synthetic workloads simulating typical editing patterns.

Key algorithms include type unification, bidirectional type checking with constraint generation, normalisation for dependent types.

6.4 Success Criteria and Evaluation Plan

6.4.1 Success Criteria

1. Correctly type check a language with dependent types, including Pi types, Sigma types and inductive types
2. Demonstrate measurable performance improvement for incremental changes vs full re-elaboration

6.4.2 Evaluation Plan

- Correctness: test suite of programs including simple proofs of equality, existential and universal quantification (dependent pair and arrow types).
- Performance and scalability: Benchmarks measuring re-elaboration time for:
  - local definition changes
  - type signature modifications
  - cascading dependency updates

  on sample programs, generated by a computer, of at least 1000 lines of code

6.5 Work Plan

6.5.1 Michaelmas Term

- 9 Oct – 22 Oct: Read Salsa documentation, study dependent type implementation, implement toy examples, design language syntax and semantics
- 23 Oct – 6 Nov: Implement elaborator for dependent types over a query-based interface
- 7 Nov – 19 Nov: Complete elaborator for dependent types, including identity types, dependent pairs; write example programs
- 20 Nov – 3 Dec: Decompose elaborator, and other stages of compilation process into fine-grained queries

6.5.2 Lent Term

- 22 Jan – 4 Feb: Implement incrementalisation with change detection, cache invalidation, and dependency tracking, utilising Salsa
- 5 Feb – 18 Feb: Complete incrementalisation, with source-based tracking
- 19 Feb – 4 Mar: Optimise incrementalisation, query granularity and dependency tracking
- 5 Mar – 18 Mar: Evaluation and performance benchmarks, with synthetic programs

6.5.3 Easter Term

- 30 Apr – 15 May: Dissertation

6.6 Resources Declaration

- Personal laptop (32 GB RAM, Intel Ultra 7 155) for development
- Backup strategy: Git repository on GitHub
- Contingency plan: Backup laptop available

6.7 Ethics Approval

Not required as no human participants or personal data involved.